# Etapa 4 - AST y Chequeo de Sentencias de MiniJava

Compiladores e Intérpretes — UNS, 2026

Implementación de la construcción de los Árboles de Sintaxis Abstracta (AST) de métodos y constructores y del chequeo de sentencias del analizador semántico de MiniJava. Se extiende el trabajo de la Etapa 3 (Tabla de Símbolos y chequeo de declaraciones) para que, durante el parseo, se construya el AST del cuerpo de cada unidad y, una vez chequeadas y consolidadas las declaraciones, se recorran esos cuerpos resolviendo nombres, chequeando tipos y validando la información estructural. No se implementan logros de la etapa; el alcance es el de la semántica base de sentencias.

## 1 Objetivo y logros

- Construir el AST del bloque de cada método y constructor y asociarlo a su entrada de la Tabla de Símbolos (TS).
- Chequear los cuerpos después del chequeo y la consolidación de declaraciones: resolución de nombres, chequeo de tipos, información estructural, ámbitos léxicos de variables locales, contexto estático y conformidad de tipos de MiniJava.
- Reportar el primer error con un mensaje descriptivo y el código `[Error:lexema|nroLinea]`, o `[SinErrores]` si el programa es válido.

Se mantienen los logros de etapas anteriores que siguen vigentes (atributos inicializados, operadores posfijos, visibilidad mejorada, manejo de archivos eficiente y genericidad avanzada de las Etapas 2 y 3). **No se intentan logros de la Etapa 4**: no se implementan MiniLambdas, `for`, ternario, `instanceof`, `super`, retorno en todos los caminos, floats, métodos genéricos, multi-detección de errores ni `main`. La sintaxis aceptada sigue siendo la de la Etapa 3; los controles agregados son sólo los de la semántica base de sentencias.

## 2 Esquema general

El módulo principal recibe el archivo fuente, lo asocia al manejador de archivos y este al léxico. Luego crea la `TablaDeSimbolos` (que ya carga `Object`, `String` y `System`) y el `AnalizadorSintactico` con dependencia al léxico y a la TS. El sintáctico, mientras valida la sintaxis, ejecuta las acciones semánticas que insertan las entidades en la TS, realizan los controles de la primera pasada y construyen el AST del cuerpo de cada unidad. Al finalizar el parseo, el módulo principal ejecuta el chequeo de declaraciones (`estaBienDeclarada`), la consolidación (`consolidar`) y, por último, el chequeo de cuerpos (`chequearCuerpos`). Si no hay errores informa el mensaje de éxito; ante un error léxico, sintáctico o semántico captura la excepción correspondiente, reporta el mensaje y el código de error, y finaliza la ejecución.

## 3 Estructura del proyecto

```
compilador-minijava/
|-- Compilar.sh / Compilar.bat          Compila y genera Compilador.jar
```

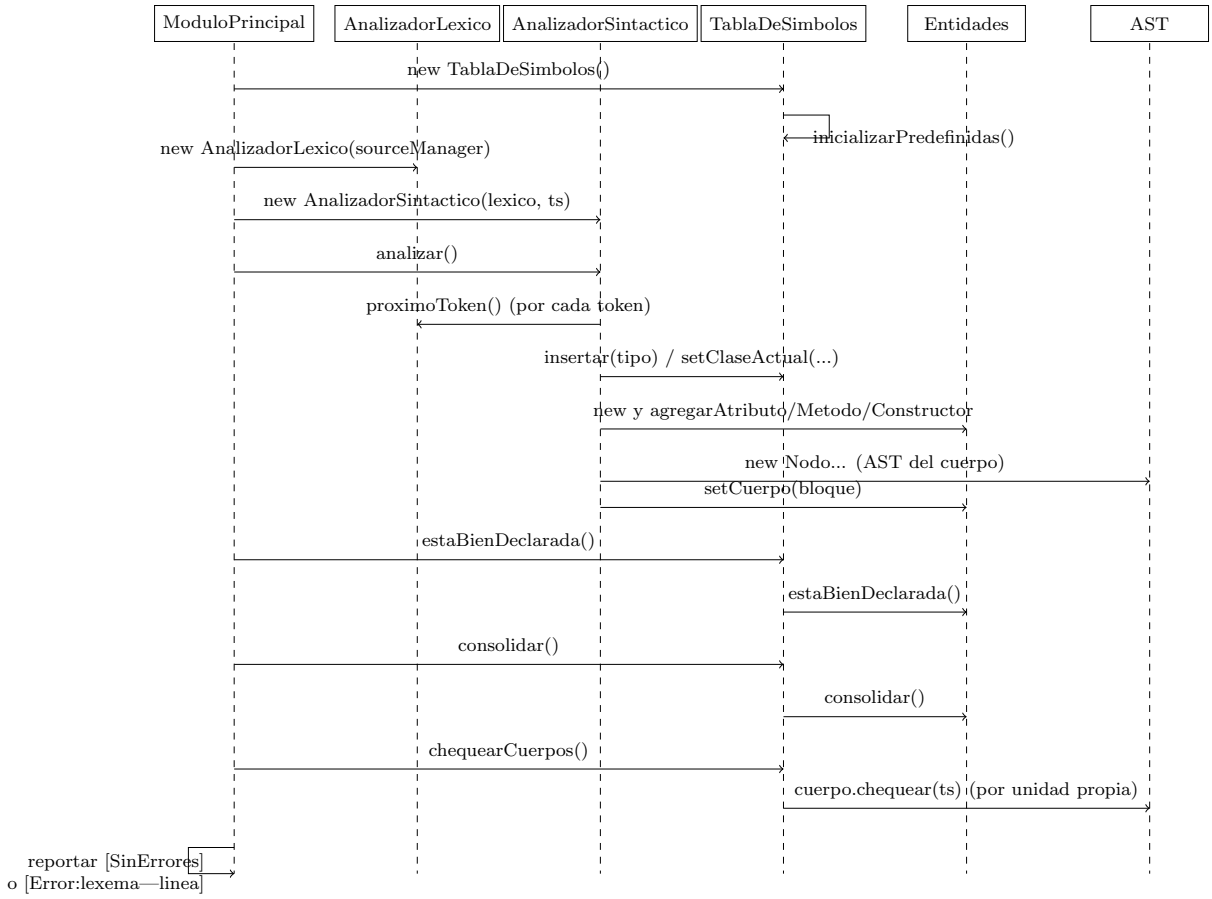



Figure 1: Diagrama de secuencia del análisis de la Etapa 4.

```
|-- CorrerTests.sh / CorrerTests.bat    Compila y ejecuta los testers JUnit
|-- lib/                                Jars de JUnit 4 y Hamcrest (se autodescargan)
|-- src/main/java/
|   |-- moduloprincipal/
|   |   \-- ModuloPrincipal.java        Modulo principal (interfaz con el usuario)
|   |-- analizadorlexico/               Etapa 1 (sin cambios)
|   |-- analizadorsintactico/
|   |   |-- AnalizadorSintactico.java   Parser + acciones semanticas (TS y AST)
|   |   \-- ExcepcionSintactica.java    Error sintactico
|   |-- analizadorsemantico/            Tabla de Simbolos y chequeos
|   |   |-- TablaDeSimbolos.java        Tabla global, predefinidas, ambitos, cuerpos
|   |   |-- Entidad.java                Entidad declarable (abstracta)
|   |   |-- Unidad.java                 Metodo/Constructor (abstracta)
|   |   |-- Clase.java / Interfaz.java  Tipos declarados
|   |   |-- Metodo.java / Constructor.java / Atributo.java / Parametro.java
|   |   |-- VariableLocal.java / Bloque.java
|   |   |-- Tipo.java y subclases       Modelo de tipos (primitivo, void, T, arreglo, ref)
|   |   |-- InfoCheck.java              Tipo + asignable + llamada
|   |   |-- SemanticaUtil.java          Conformidad, vistas genericas, contexto
|   |   \-- ExcepcionSemantica.java     Error semantico
|   |-- analizadorsemantico/ast/        NUEVO: nodos del AST (expresiones y sentencias)
|   \-- sourcemanager/                  Etapa 1 (sin cambios)
|-- src/test/java/test/                 Testers JUnit provistos por la catedra
\-- resources/
    |-- sinErrores/                     Casos de prueba sin errores
    \-- conErrores/                     Casos de prueba con errores
```

# 4 Diseño e implementación de los AST

Cada tipo de nodo es una clase; la herencia modela las relaciones "es-un". Los nodos conservan tokens para poder reportar el error con el lexema y la línea correctos.

## 4.1 Gramática de diseño y gramática de construcción

Los tipos de nodo del AST se *diseñan* a partir de la gramática original del lenguaje (los constructores sintácticos que tienen significado), pero el AST se *construye* durante el análisis sintáctico con una Especificación Dirigida por la Traducción (EDT) sobre la gramática LL(1) que efectivamente implementa el parser. La cátedra pide explícitamente esta separación: el diseño se guía por la gramática original, mientras que la construcción se hace sobre la gramática del parser.

Las diferencias entre ambas gramáticas se resuelven en el parser, no en el diseño:

- **Recursión a izquierda**: `ExpresionCompuesta` y `Referencia` son recursivas a izquierda en la gramática original. La versión LL(1) (`ECResto`, `RResto`) las reconoce con recursión a derecha y las arma con un acumulador, produciendo árboles asociativos a izquierda.
- **Sentencias de expresión**: la gramática original distingue `Sentencia ::= Asignacion ;` y `Sentencia ::= Llamada ;`, pero ambas son alias de `Expresion`. La transformación LL(1) las unifica en `Sentencia ::= Expresion ;`; por eso hay un único `NodoSentenciaExpresion`, que clasifica la forma durante el chequeo (`NodoExpresionAsignacion` vs. `InfoCheck.esLlamada()`).

Correspondencia entre producciones de la gramática original y nodos del AST:

| Produccion (gramatica original) | Nodo |
|---|---|
| AccesoVar ::= idMetVar | NodoVar |
| LlamadaMetodo ::= idMetVar ArgsActuales | NodoLlamada |
| LlamadaMetodoEstatico | NodoLlamadaEstatica |
| LlamadaConstructor ::= new TipoRef Args | NodoNewObjeto |
| CrearArreglo ::= new TipoBase Dimensiones | NodoNewArreglo |
| ExpresionParentizada | NodoExpresionParentizada |
| Primario this / stringLiteral | NodoThis / NodoLiteral |
| Primitivo | NodoLiteral |
| Expresion con OperadorAsignacion | NodoExpresionAsignacion |
| ExpresionCompuesta con OperadorBinario | NodoExpresionBinaria |
| ExpresionBasica con OperadorUnario | NodoExpresionUnaria |
| PosfijoOpcional (solo en la LL(1)) | NodoExpresionPosfijo |
| VarEncadenada ::= . idMetVar | NodoEncadenadoAtributo |
| MetodoEncadenado ::= . idMetVar Args | NodoEncadenadoLlamada |
| AccesoArreglo ::= [ Expresion ] | NodoEncadenadoArreglo |
| Bloque / Sentencia ; | NodoBloque / NodoSentenciaVacia |
| VarLocal / Return / If / While | NodoVarLocal / NodoReturn / NodoIf / NodoWhile |
| Sentencia ::= Asignacion ; / Llamada ; | NodoSentenciaExpresion |

En síntesis: los **tipos de nodo** reflejan los constructores significativos de la gramática original; la **forma del árbol** es la que produce el parseo LL(1) implementado.

## 4.2 Jerarquía de expresiones

```
NodoExpresion (abstracta)
|-- NodoLiteral
|-- NodoThis
|-- NodoVar
|-- NodoLlamada
|-- NodoLlamadaEstatica
|-- NodoNewObjeto
```

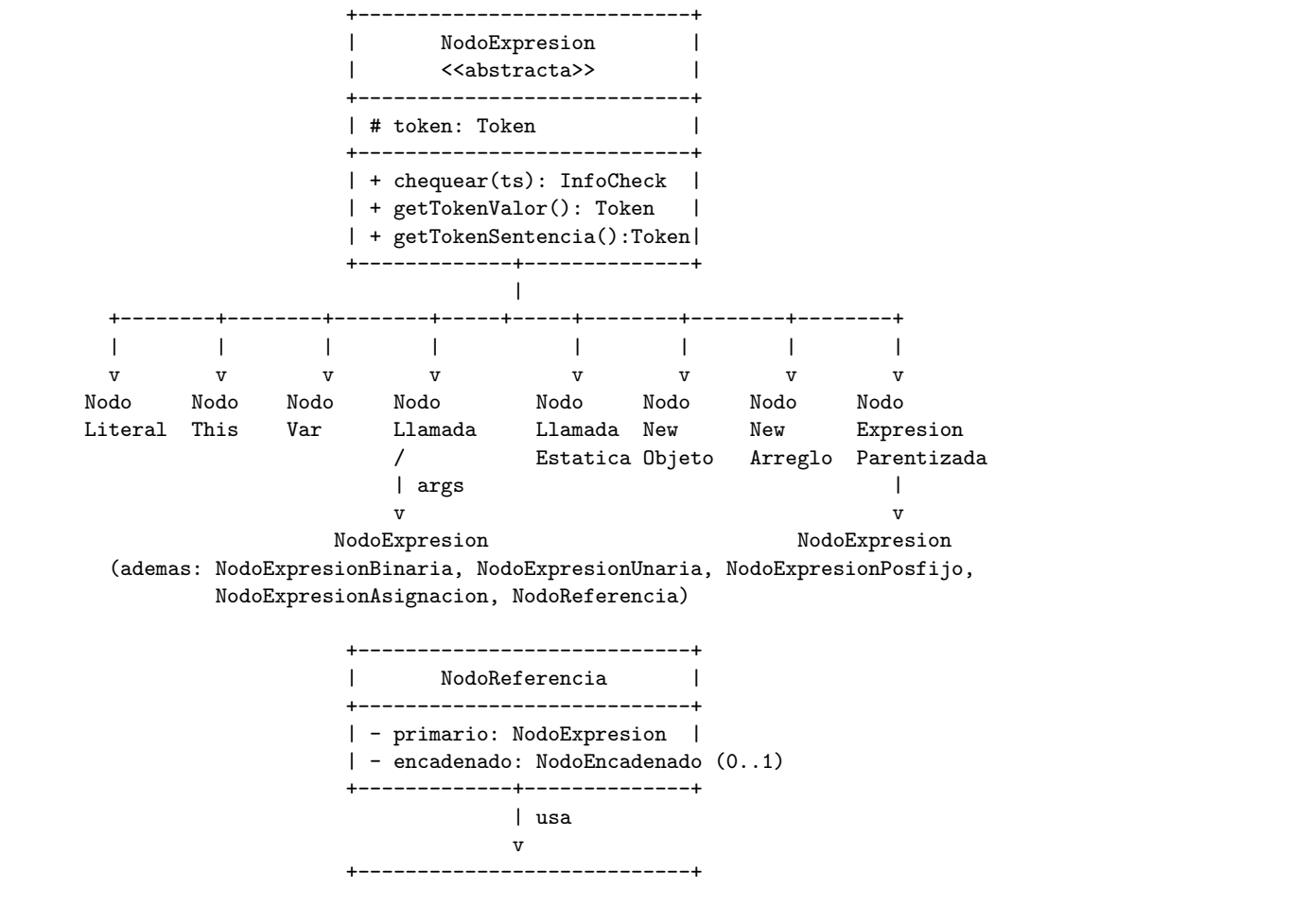
```
|-- NodoNewArreglo
|-- NodoExpresionParentizada
|-- NodoExpresionBinaria
|-- NodoExpresionUnaria
|-- NodoExpresionPosfijo
|-- NodoExpresionAsignacion
\-- NodoReferencia --------------------+
                                       | usa
NodoEncadenado (abstracta) <-----------+
|-- NodoEncadenadoAtributo
|-- NodoEncadenadoLlamada
\-- NodoEncadenadoArreglo
```

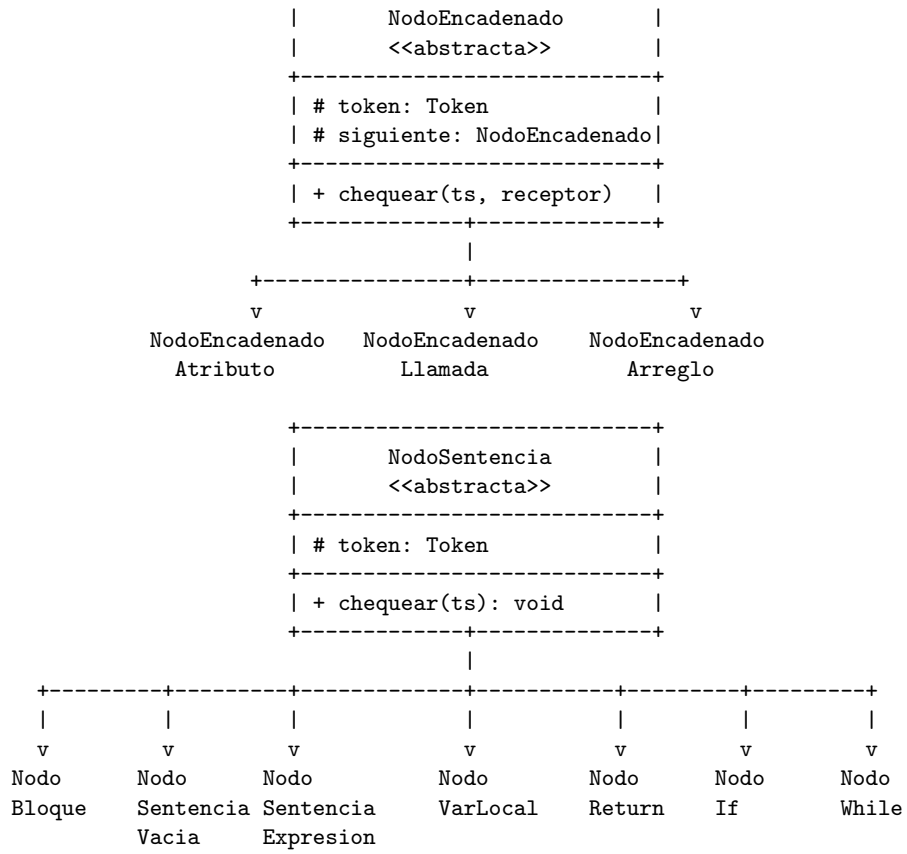
`NodoReferencia` modela `Primario RResto`: un primario seguido de un encadenado opcional. Cada eslabón recibe el tipo estático del receptor, lo controla y delega en el siguiente. Esto cubre accesos como `crearPersona().edad`, `matriz[0].length` o `caja.obtener().toString()`.

### 4.3 Jerarquía de sentencias

```
NodoSentencia (abstracta)
|-- NodoBloque
|-- NodoSentenciaVacia
|-- NodoSentenciaExpresion
|-- NodoVarLocal
|-- NodoReturn
|-- NodoIf
\-- NodoWhile
```

### 4.4 Diagrama de clases






### 4.5 Métodos de chequeo

- Toda expresión implementa `InfoCheck chequear(TablaDeSimbolos ts)`.
- Toda sentencia implementa `void chequear(TablaDeSimbolos ts)`.
- Toda expresión aplica el patrón de **responsabilidad + delegación**: controla lo propio y delega en sus hijos.
- Cada expresión expone además `getTokenValor()` (token que la representa en contextos que exigen un tipo: condición, retorno, argumento) y `getTokenSentencia()` (token señalado cuando no es admisible como sentencia).

## 5 Construcción de los AST durante el análisis sintáctico

Se modificó `AnalizadorSintactico` para que las acciones semánticas construyan nodos, siguiendo una EDT sobre la gramática LL(1) que el parser ya implementaba. Los tipos de nodo provienen del diseño guiado por la gramática original; la traducción de las reglas recursivas a izquierda se hace con acumuladores en el parser.

### 5.1 Sentencias

| Produccion | Nodo construido |
|---|---|
| `Bloque` | `NodoBloque` |
| `Sentencia ;` | `NodoSentenciaVacia` |
| `Expresion ;` | `NodoSentenciaExpresion` |
| `var id = ExpresionCompuesta` | `NodoVarLocal` |
| `return ExpresionOpcional` | `NodoReturn` |
| `if (Expresion) Sentencia else Sentencia` | `NodoIf` |

`while (Expresion) Sentencia` `NodoWhile`

### 5.2 Expresiones

- `expresion()` construye `NodoExpresionAsignacion` si aparece `=`.
- `expresionCompuesta()` / `eCResto()` construyen `NodoExpresionBinaria` asociando a izquierda, respetando el único nivel de binarios de la gramática.
- `expresionBasica()` construye `NodoExpresionUnaria` y `NodoExpresionPosfijo`.
- `referencia()` construye `NodoReferencia` cuando hay encadenado y devuelve el primario directo cuando no lo hay.
- `rResto()` encadena `NodoEncadenadoAtributo`, `NodoEncadenadoLlamada` y `NodoEncadenadoArreglo`.
- `primario()` construye `NodoThis`, `NodoLiteral` (string), `NodoVar`, `NodoLlamada`, `NodoNewObjeto`, `NodoNewArreglo`, `NodoLlamadaEstatica` o `NodoExpresionParentizada`.

### 5.3 Asociación a la TS

Al terminar de parsear el bloque de un método o constructor, el nodo se asocia a la unidad antes de insertarla en la clase:

```
NodoBloque cuerpo = bloque();
Metodo metodo = new Metodo(tokenNombre, tipoRetorno, parametros, esEstatico, ts);
metodo.setCuerpo(cuerpo);
agregarMetodo(metodo);
```

## 6 Chequeo de sentencias

El chequeo se ejecuta **después** de `estaBienDeclarada()` y `consolidar()`. `TablaDeSimbolos.chequearCuerpos()` recorre las clases y, para cada una, sólo las unidades propias; así un cuerpo heredado no se rechequea en el descendiente.

### 6.1 Resolución de nombres

Para un identificador sin receptor (`NodoVar`) se busca en este orden:

1. variable local visible (bloque actual hacia contenedores),
2. parámetro de la unidad actual,
3. atributo de la clase actual (incluidos los heredados consolidados).

Si se selecciona un atributo en contexto estático se reporta error. Si no se encuentra, “la variable X no fue declarada”.

Para llamadas implícitas (`NodoLlamada`) se busca la clave `(nombre, aridad)` en los métodos de la clase actual, incluidos los heredados. En contexto estático sólo se admiten métodos estáticos.

Para receptores explícitos se usa el **tipo estático** del receptor:

- referencias: se busca el miembro en la clase/interfaz y se aplica la sustitución genérica correspondiente a la vista;

- interfaces: además de sus propios métodos, ofrecen los métodos de instancia de `Object` (por ejemplo `toString`), pero no sus métodos estáticos ni los de la clase que las implementa;
- arreglos: sólo `length` (sólo lectura) y `toString()` sin argumentos;
- primitivas, `null`, `void` o parámetros de tipo no instanciados no permiten seleccionar miembros.

## 6.2 Chequeo de tipos

- Operadores: `int` para `+ - * / %` y comparaciones; `boolean` para `&& || !`; `== !=` exigen conformidad en alguna dirección; sin promociones ni concatenación con `+`.
- Asignación: el lado izquierdo debe ser asignable y el derecho debe conformar con el izquierdo. El resultado tiene el tipo del destino y no es asignable.
- Llamadas y creaciones: se controla aridad y que cada argumento conforme con el formal correspondiente.
- Creación de arreglos: cada dimensión debe ser `int`; el tipo base debe ser válido y, si es genérico, no puede omitirse el argumento.
- `var`: el inicializador se chequea antes de declarar la variable; no se admite `void` ni `null` como origen de inferencia.
- `if` / `while`: la condición debe ser `boolean`.
- `return`: en `void` sólo `return;`; en métodos con retorno, `return e;` con conformidad; en constructores sólo `return;`.

## 6.3 Información estructural

`InfoCheck` porta `asignable` y `llamada`. Reglas implementadas:

- locales, parámetros, atributos, componentes de arreglo: asignables;
- `length`: no asignable;
- literales, `this`, paréntesis, resultados de operadores, asignaciones, creaciones: no asignables;
- una llamada a método o la creación de un objeto son admisibles como sentencia de llamada; una creación de arreglo no lo es;
- una expresión parentizada conserva el tipo pero **pierde** ambas propiedades (`(x) = 3`, `(x = 3);`, `(metodo());`, `(objeto).metodo();` se rechazan);
- una sentencia de expresión sólo es válida si es una asignación o termina en una llamada (por eso `crearPersona().edad;` se rechaza aunque su tipo sea asignable).

## 6.4 Ámbitos y declaraciones locales

- Al entrar a un bloque se crea un ámbito; al salir se restaura el anterior.
- Una local no puede repetir el nombre de un parámetro de la unidad ni el de una local visible en el bloque actual o en un contenedor.
- Puede ocultar un atributo.

- Bloques hermanos pueden reutilizar nombres.
- El inicializador no ve a la variable que se está declarando.
- Se rechaza una declaración local como cuerpo directo de `if`, `else` o `while`.
- Se chequean todas las sentencias, incluso después de un `return` (no se rechaza código inalcanzable).

### 6.5 Contexto estático

- No se permite `this` ni el acceso implícito a atributos o métodos de instancia.
- Sí se permite acceder a miembros de instancia mediante un receptor explícito obtenido de un parámetro, local, llamada o creación.
- Se rechaza el parámetro de tipo de la clase en encabezados y cuerpos de métodos estáticos (incluido dentro de arreglos o argumentos genéricos).

### 6.6 Conformidad de MiniJava

Se respetaron explícitamente las diferencias con Java:

- genéricos y arreglos invariantes;
- sin tipos raw (fuera de la excepción de herencia sin argumento);
- sin cotas para parámetros de tipo;
- herencia genérica sin argumento conserva el parámetro original; un parámetro conservado se identifica por su declaración de origen, de modo que dos parámetros que comparten letra pero provienen de declaraciones distintas no conforman entre sí;
- arreglos conforman con `Object`, pero no entre arreglos de distinto tipo base;
- `null` conforma con referencias, parámetros y arreglos, no con primitivas;
- `void` no conforma con tipos de valor.

## 7 Entidades predefinidas

Antes de analizar el programa la tabla contiene:

- `Object`, que declara `String toString()` (de instancia) y `static void debugPrint(int)`; tiene un constructor por defecto;
- `String`, tipo de los literales de cadena, que hereda de `Object`, tiene constructor por defecto y no declara métodos propios;
- `System`, que hereda de `Object` y declara los métodos estáticos `read()`, `printB(boolean)`, `printC(char)`, `printI(int)`, `printS(String)`, `println()`, `printBln(boolean)`, `printCln(char)`, `printIln(int)` y `printSln(String)`.

Toda clase sin relación explícita extiende `Object` y hereda sus miembros; una clase que implementa una interfaz también es un subtipo de `Object`. Los arreglos ofrecen únicamente `length` y el `toString()` de `Object`. Una interfaz no puede declarar un método con la misma clave que un método de instancia de `Object` con signatura distinta.

# 8 Reporte de errores

Al detectarse un error se reporta y finaliza la ejecución. El token asociado sigue las pautas de la cátedra: se señala el token del nodo que tiene el error. En particular:

- errores de creación: la palabra reservada `new`;
- errores de selección de miembro o de indexación: el selector `.` o el corchete `[`;
- errores de tipo de un índice, tamaño, retorno o condición: la subexpresión ofensora (por ejemplo el índice, el tamaño o el valor retornado);
- llamadas implícitas no resueltas: el nombre del método;
- argumentos de una llamada con receptor explícito: el nombre del método;
- asignaciones y operadores: el operador correspondiente.

```
Error Semantico en linea <N>: <mensaje>
[Error:<lexema>|<N>]
```

# 9 Formato de salida

## 9.1 Análisis exitoso

```
Compilacion Exitosa
[SinErrores]
```

## 9.2 Análisis con errores

Mensaje descriptivo seguido del código de error, por ejemplo:

```
Error Semantico en linea 5: el tipo boolean no conforma con el tipo de retorno int
[Error:true|5]
```

# 10 Compilación y ejecución

No se usa ninguna herramienta de build: solo se necesitan `javac`, `jar` y `java` de cualquier JDK 21 o superior (probado con JDK 25).

```
./Compilar.sh             # compila y genera Compilador.jar  (Windows: Compilar.bat)
java -jar Compilador.jar resources/sinErrores/semIICorrecto01.java
./CorrerTests.sh          # compila y ejecuta los testers JUnit  (Windows: CorrerTests.bat)
```

## 10.1 Comandos manuales equivalentes

Compilar (desde `compilador-minijava/`):

```
javac -d build/classes \
  src/main/java/moduloprincipal/*.java \
  src/main/java/analizadorlexico/*.java \
  src/main/java/analizadorsintactico/*.java \
  src/main/java/analizadorsemantico/*.java \
  src/main/java/analizadorsemantico/ast/*.java \
  src/main/java/sourcemanager/*.java
jar cfe Compilador.jar moduloprincipal.ModuloPrincipal -C build/classes .
```

Correr los tests (desde `compilador-minijava/`, con JUnit 4.13.2 y Hamcrest 1.3 en `lib/`):

```
javac -cp "build/classes:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar" \
  -d build/test-classes src/test/java/test/*.java

java -cp "build/classes:build/test-classes:lib/junit-4.13.2.jar:\
lib/hamcrest-core-1.3.jar" org.junit.runner.JUnitCore \
  test.TesterDeCasosSinErrores test.TesterDeCasosConErrores
```

En Windows el separador del classpath es `;` en lugar de `:`.

## 11 Tests

Se incluyen los casos provistos por la cátedra y casos propios desarrollados para esta etapa (el diseño de los casos de prueba es responsabilidad del alumno). Resultado actual: **404/404 casos OK**.

Los testers descubren automáticamente todos los archivos de `resources/sinErrores/` y `resources/conErrores/`. La batería utilizada es la de la cátedra (archivos `TAMBIENBORRAR_*.txt`) junto con los casos base `semIIError*.java` y `semIICorrecto*.java`. Los archivos antiguos de convención previa que contradecían a la batería nueva se apartaron en `tests_descartados/` para que los testers no los descubran.

Los casos sin errores cubren, entre otros: predefinidas y uso de `Object.debugPrint` y `toString` (en clases, interfaces, arreglos y cadenas); creación de `Object` y `String`; herencia encadenada; redefinición exacta; sobrecarga por aridad; implementación de interfaces (simples, con extensión y genéricas); clases genéricas con parámetros en atributos, retornos y parámetros; herencia genérica con y sin argumento; vistas genéricas en accesos y llamadas; arreglos multidimensionales; `this`, `length`, `if`, `while`, `null`; métodos estáticos; paréntesis y pérdida de propiedades estructurales; y declaraciones locales con visibilidad por bloques.

Los casos con errores cubren, entre otros: nombres de tipo repetidos; redeclaración de las predefinidas; relaciones de herencia inválidas o circulares; atributos y métodos repetidos; redefiniciones con firma distinta; contratos de interfaces; tipos inválidos; argumentos genéricos inválidos (incluidos anidados); tipos raw; operadores y asignaciones con tipos incompatibles; resolución de nombres y ámbitos; contexto estático; información estructural de sentencias (asignabilidad y forma de llamada); indexación y creación de arreglos; y retornos inválidos.

## 12 Fuera de alcance (logros)

No se implementaron los logros de la etapa: MiniLambdas, `for`, retorno en todos los caminos sin código muerto, floats y coerción, ternario, genericidad avanzada (inferencia por diamante completa), `instanceof`, visibilidad, `super`, métodos genéricos, multi-detección de errores, atributos inicializados, postfijos como sentencia ni `main`. La sintaxis aceptada sigue siendo la de la Etapa 3; los controles agregados son sólo los de la semántica base de sentencias.

## 13 Anexo A - Diferencias de sintaxis respecto de la gramática original

El parser implementado (heredado de las Etapas 2 y 3) acepta algunos constructos que no figuran en el archivo de la gramática original ni en la semántica base. Se listan para dejar explícita la diferencia; el chequeo de sentencias de la Etapa 4 base no los trata de forma especial.

| Constructo | Gramatica original | Gramatica implementada (LL1) |
|---|---|---|
| public / private | no figuran | VisibilidadOpcional |
| Posfijos ++ / -- | no figuran | PosfijoOpcional |

```
Argumentos genericos anidados no                instanciadoOParametrico recursivo
Notacion diamante new Caja<>  no                admitida si la clase es generica
```

Estas extensiones provienen de los logros de las Etapas 2 y 3. Los controles semánticos de esta entrega que las alcanzan son los mínimos: por ejemplo, el posfijo se admite sólo dentro de expresiones y sobre operandos `int` asignables, y `new Caja<>()` no infiere el argumento (eso corresponde al logro de genericidad avanzada).